

At scale with XPK

Contents

- Cluster Toolkit replacement
- Overview of the workflow
- 1. Prerequisites
- 2. One-time environment setup
- 3. Build the MaxText Docker image
- 4. Run your first MaxText job
- 6. Running with Ahead-of-Time (AOT) Compilation
- 7. Managing and monitoring your job

Предупреждение

Это руководство предназначено только для обеспечения совместимости с предыдущими версиями. Рекомендуемый способ развертывания новых кластеров GKE — [запуск MaxText с помощью Cluster Toolkit](#). XPK устарел, и его не следует использовать для подготовки новых кластеров или оркестрации рабочих нагрузок.

Прямую ссылку на команды Cluster Toolkit см. в разделе [Масштабирование с помощью Cluster Toolkit](#). Приведенные ниже команды XPK сохранены только для существующих развертываний.

Замена Cluster Toolkit

Для новых развертываний см. [Запуск MaxText с помощью Cluster Toolkit](#).

В этом руководстве описан старый способ запуска MaxText Engine (GKE) с использованием **Accelerated Processing Kit (XPK)**. Полную информацию об XPK можно найти в [официальном репозитории XPK](#).

 [Последние](#) ▼

Обзор рабочего процесса

Процесс состоит из двух основных этапов. Сначала вы упакуете приложение MaxText и его зависимости в автономный образ Docker. Это можно сделать на локальном компьютере или в любой среде, где установлен Docker. Затем с помощью инструмента командной строки ХРК вы организуете развертывание этого образа в качестве обучающего задания в кластере GKE, оснащенный ускорителями (TPU или графическими процессорами).

ХРК упрощает управление кластером и отправку заданий, выполняя такие задачи, как загрузка образа Docker в Artifact Registry и планирование рабочей нагрузки в кластере.

```
+-----+ +-----+ +-----+
| | | | |
| Ваш компьютер для разработки +-----> Реестр артефактов +-----> Кластер G
| (где угодно с помощью Docker) | | (хранит ваш образ) | |(с ускорителями)|
| | | | |
| 1. Сборка MaxText Docker | | 2. Загрузка ХРК | | 3. Запуск задания ХРК |
| Изображение | | изображение для вас | | с использованием изображения |
+-----+ +-----+ +-----+
```

1. Предварительные требования

Прежде чем начать, убедитесь, что у вас установлены необходимые инструменты и настроены разрешения.

Необходимые инструменты

- Python >= 3.12 с `pip` и `venv`.
- Google Cloud CLI (`gcloud`): Установите его [здесь](#), а затем запустите `gcloud init`.
- **kubectl**: Инструмент командной строки Kubernetes.
- **Docker**: Следуйте [инструкциям по установке](#) и [шагам по настройке Docker без sudo](#).

Разрешения GCP

Для используемого вами проекта в вашей учетной записи Google Cloud должны быть назначены следующие роли IAM:

- Администратор реестра артефактов
- Администратор вычислительных ресурсов
- Администратор Kubernetes Engine
- Администратор логирования
- Администратор мониторинга
- Пользователь служебной учетной записи
- Администратор хранилища
- Администратор Vertex AI

2. Однократная настройка среды

Эти команды настраивают локальную среду для подключения к сервисам Google Cloud.

1. Аутентификация gcloud

```
gcloud auth login
```

2. Установите плагин GKE auth

```
sudo apt-get update && sudo apt-get install google-cloud-cli-gke-gcloud-
```

3. Настройка учетных данных Docker

```
gcloud auth configure-docker
```

3. Создайте образ MaxText Docker

Инструкции по созданию образа MaxText Docker см. в [официальной документации](#)

4. Запустите первое задание MaxText

В этом разделе предполагается, что у вас уже есть кластер GKE с узлами TPU или GPU.

! Важный

This guide focuses on submitting workloads to an existing cluster. Cluster creation and management is a separate topic. For a comprehensive guide on all `xpk` commands, including `xpk cluster create`, please refer to the [official XPK documentation](#).

1. Set your configuration

Set up the following environment variables to configure your training run. Replace placeholders with your actual values.

```
# -- Google Cloud Configuration --
# Your GCP project ID. Find it on the [Cloud Console Dashboard](https://
# If you've already set it in your local config, you can retrieve it via
# gcloud config get-value project
export PROJECT_ID=<GCP project ID>

# The GCP location (listed as "Location" in the UI) and name of your
# TPU-enabled (or GPU-enabled) GKE cluster. Both can be found on the
# [Cloud Console](https://console.cloud.google.com/kubernetes/list).
export ZONE=<GCP location> # e.g., 'us-central1' or 'us-central1-a'
export GKE_CLUSTER=<cluster name>

# -- Workload Configuration --
# An arbitrary string to identify this specific run.
# Note: Kubernetes requires workload names to be valid DNS labels (lower
export RUN_NAME="maxtext-run-$(date +%Y%m%d-%H%M%S)"

# Number of TPU slices (for TPU clusters) or number of nodes (for GPU cl
export NUM_SLICES=1

# -- MaxText & Storage Configuration --
# Use a GCS bucket you own to store logs and checkpoints. Ideally in the
# region as your TPUs to minimize latency and costs.
# You can list your buckets and their locations in the
# [Cloud Console](https://console.cloud.google.com/storage/browser).
export BASE_OUTPUT_DIRECTORY=<GCS_BUCKET> # e.g., gs://<my-bucket>/max
export DATASET_PATH="gs://<DATASET_PATH>/"
```

2. Configure gcloud CLI

```
gcloud config set project ${PROJECT_ID?}
gcloud config set compute/zone ${ZONE?}
```

A Note on multi-slice and multi-node runs

The examples below run on a single TPU slice (`--num-slices=1`) or a small number of GPU nodes (`--num-nodes=2`). To scale your job to a larger, multi-host configuration, you simply increase the `NUM_SLICES` value.

For instance, to run a job across **four TPU slices**, you would change `export NUM_SLICES=1` to `export NUM_SLICES=4`. This tells XPK to allocate four `v5litepod-256` slices and orchestrate the training job across all of them as a single workload. Similarly, for GPUs, you would increase the value.

Note

If you installed `xpk` inside a Python virtual environment (`venv`), make sure to reactivate your virtual environment (e.g., `source <VENV_NAME>/bin/activate`) in any new terminal sessions before running `xpk` commands. Otherwise, you will encounter a `Command xpk not found` error.

3. Create the workload (run the job)

◦ On your TPU cluster:

```
xpk workload create \
  --cluster ${GKE_CLUSTER?} \
  --workload ${RUN_NAME?} \
  --base-docker-image maxtext_base_image \
  --tpu-type v5litepod-256 \
  --num-slices ${NUM_SLICES?} \
  --command "python3 -m maxtext.trainers.pre_train.train run_name=${RUN_NAME?}"
```

◦ On your GPU cluster:

```
xpk workload create \
  --cluster ${GKE_CLUSTER?} \
  --workload ${RUN_NAME?} \
  --base-docker-image maxtext_base_image \
  --device-type h100-80gb-8 \
  --num-nodes ${NUM_SLICES?} \
  --command "python3 -m maxtext.trainers.pre_train.train run_name=${RUN_NAME?}"
```

6. Running with Ahead-of-Time (AOT) Compilation

Ahead-of-Time (AOT) compilation can significantly reduce the startup time of your training job by pre-compiling the JAX training step for a specific hardware topology. The workflow involves generating a compiled artifact, including it in your Docker image, and then telling MaxText to use it at runtime.

Step 1: Generate the AOT artifact

First, run `train_compile.py` script to create the compiled artifact for the specified TPU topology.

```
export WORKLOAD_NAME=<RUN_NAME>
export TPU_TYPE="your-tpu-type" # e.g. "v5p-32"
export NUM_SLICES=your-num-slices # e.g. 1
export PER_DEVICE_BATCH_SIZE=your-batch-size # e.g. 1
export USER=your-username

python3 -m maxtext.trainers.pre_train.train_compile \
  compile_topology=${TPU_TYPE?} \
  compile_topology_num_slices=${NUM_SLICES?} \
  compiled_trainstep_file=maxtext_${TPU_TYPE?}_aot.pickle \
  per_device_batch_size=${PER_DEVICE_BATCH_SIZE?}
```

This will create a file named `maxtext_${TPU_TYPE?}_aot.pickle` in your MaxText root directory.

Step 2: Re-build and upload your Docker image

The AOT artifact must be included in your Docker image. The `docker_upload_runner.sh` script automatically copies the contents of your MaxText directory into the image:

```
export CLOUD_IMAGE_NAME=<IMAGE_NAME>

bash src/dependencies/scripts/docker_upload_runner.sh CLOUD_IMAGE_NAME=${CLO
```

Step 3: Create the XPK workload wit

artifact

Now, submit the workload as before, but add the `compiled_trainstep_file` argument to the command and use the custom Docker Image we built in step 2. This tells MaxText to load the pre-compiled step instead of compiling it at runtime.

```
export BASE_OUTPUT_DIR=your-output-dir # e.g. gs://your-output-bucket/
export DATASET_PATH=your-dataset-path # e.g. gs://your-dataset-bucket/
export CLUSTER_NAME=your-cluster-name
export PROJECT_ID=your-project-id

xpk workload create \
  --cluster ${CLUSTER_NAME?} \
  --workload ${WORKLOAD_NAME?} \
  --docker-image "gcr.io/${PROJECT_ID?}/${CLOUD_IMAGE_NAME?}" \
  --tpu-type ${TPU_TYPE?} \
  --num-slices ${NUM_SLICES?} \
  --command "python3 -m maxtext.trainers.pre_train.train \
    run_name=${WORKLOAD_NAME?} \
    base_output_directory=${BASE_OUTPUT_DIR?} \
    dataset_path=${DATASET_PATH?} \
    steps=100 \
    per_device_batch_size=${PER_DEVICE_BATCH_SIZE?} \
    compiled_trainstep_file=/deps/maxtext_${TPU_TYPE?}_aot.pickle"
```

Your job will now start faster by skipping the JAX compilation step on the cluster.

7. Managing and monitoring your job

- **View logs in real-time:** The easiest way to see the output of your training job is through the Google Cloud Console.
 - 1. Navigate to the **Kubernetes Engine** section.
 - 2. Go to **Workloads**.
 - 3. Find your workload (e.g., `${RUN_NAME?}`) and click on it.
 - 4. Select the **Logs** tab to view the container logs.
- **List your jobs:**

```
xpk workload list --cluster ${GKE_CLUSTER?}
```

- **Analyze output:** Checkpoints and other artifacts will be saved to the Storage bucket you specified in `BASE_OUTPUT_DIRECTORY`.  **Последние** ▼
- **Delete a job:**

```
xpk workload delete --cluster ${GKE_CLUSTER?} --workload ${RUN_NAME?}
```